

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 13

Generating Answers with an LLM

“An answer is not correct because it reads well; it is correct because it is grounded in something the retriever actually found.”

Chapter 13 — Generating Answers with an LLM

Memora's simplest answer path lives in `query.py`, a deliberately small command-line tool that predates the agentic loop covered from Chapter 15 onward. It is worth building and understanding on its own terms: every later chapter's answer generation — draft, judge, retry, distill — is this same augmentation step, called more than once and wrapped in more machinery, never a different idea.

13.1 The augmentation step — prompt, context, and question

Definition — Augmentation

The step in a RAG pipeline where retrieved chunks are combined with the user's question into a single prompt, so the language model generates its answer conditioned on that specific evidence instead of on its training data alone.

The augmentation step in `query.py` is almost aggressively simple: number the retrieved documents, join them with blank lines, and drop both the question and that joined context into one instruction string.

```
context = "\n\n".join(
    [f"Document {doc['rank']}: \n{doc['content']}" for doc in
    retrieved_docs]
)
prompt = f"""You are an expert assistant. Use the following retrieved
documents to answer the question.

Question: {query}

Context:
{context}

Provide a concise and accurate answer based on the above information."""
```

Analogy — An open-book exam

A model answering from training data alone is reciting from memory — confident, but unable to point at a source. Augmentation hands the model the actual retrieved pages and asks it to answer with them open on the desk. The answer should be traceable back to a specific page, not to whatever the model happened to remember.

Nothing about this step requires an agent, a tool call, or a framework. It requires only that the context passed to the model is exactly the evidence Chapters 11 and 12 decided was worth keeping — no more, no less. Everything more sophisticated later in the book is this same idea under load.

13.2 Choosing a hosted LLM

A RAG pipeline's generation step is a provider choice as much as a code choice. Memora's own architecture decision record weighed four options before settling on one, on criteria specific to an agentic pipeline that can call the model several times per question — not just once, the way a single-pass classic RAG script would.

Provider	Strength	Tradeoff for this project
Groq	LPU hardware — very fast, cheap, open models	Narrower model catalogue than a general API
OpenAI	Highest general quality, broad tooling	Slower and roughly 10x the cost at this call volume
Together AI	Open-source models, competitive pricing	Not evaluated as deeply for this project's needs
Local (Ollama / vLLM)	No per-call API cost	Hardware-constrained; latency depends on local GPU

The deciding factor was latency compounding: an agentic loop that retrieves, compresses, and judges may call the LLM five to ten times for a single user question, so per-call latency is not a minor detail — it is multiplied by every iteration the loop takes. Groq's LPU inference measured roughly 250 tokens per second against a typical GPU provider's 50, and because both Groq and OpenAI expose OpenAI-compatible REST APIs, the choice was never permanent: switching providers later is a `base\_url` and `api\_key` change, not a rewrite.

Anthropic's Claude models are a reasonable option on the same axis, reached through the identical OpenAI-compatible pattern once a project's priorities shift from development-loop speed toward maximum instruction-following and reasoning quality on the final, user-facing generation call — the same tradeoff Section 13.6 makes deliberately per-role rather than per-project, since a compression or judging call rarely needs the strongest, most expensive model available.

13.3 Groq with langchain-groq — fast and cheap for development

Definition — Inference provider

A hosted service that runs a model and exposes it over an API, so an application never has to own the GPU, the weights, or the serving stack itself — only the network call and the credentials.

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    api_key=os.getenv("GROQ_API_KEY"),
    model_name=os.getenv("MODEL_NAME", "llama-3.1-8b-instant"),
    temperature=0.1,
    max_tokens=2048,
    max_retries=0,
)
```

``max_retries=0`` is deliberate, not an oversight — it appears on every LLM client Memora constructs. LangChain's own retry wrapper and the project's own ``llm_invoke`` wrapper (Section 13.7) would otherwise both be retrying the same failed call, doubling backoff delays and making failures harder to diagnose. Retry belongs in exactly one place; here, it is explicitly turned off at the client so it can be owned centrally.

Model	Context window	Speed	Tool-use reliability
<code>`llama-3.1-8b-instant`</code>	128K in / 8K out	Fastest	Moderate — degrades past ~16K prompt tokens
<code>`llama-3.3-70b-versatile`</code>	128K in / 32K out	Slower	Strong — better instruction-following

Memora used the 8B model for development, where iteration speed matters more than perfect instruction-following, and identified the 70B model as the production-grade target — particularly for compression and judging stages, where faithfulness to the source text matters more than raw speed.

13.4 A simple RAG answer function, end to end

Figure 13.1 lays out the whole generation stage as one straight line, deliberately with no branching and no loop — that comes later, once Chapter 15 turns this same augmentation step into something an agent can call repeatedly with different queries.

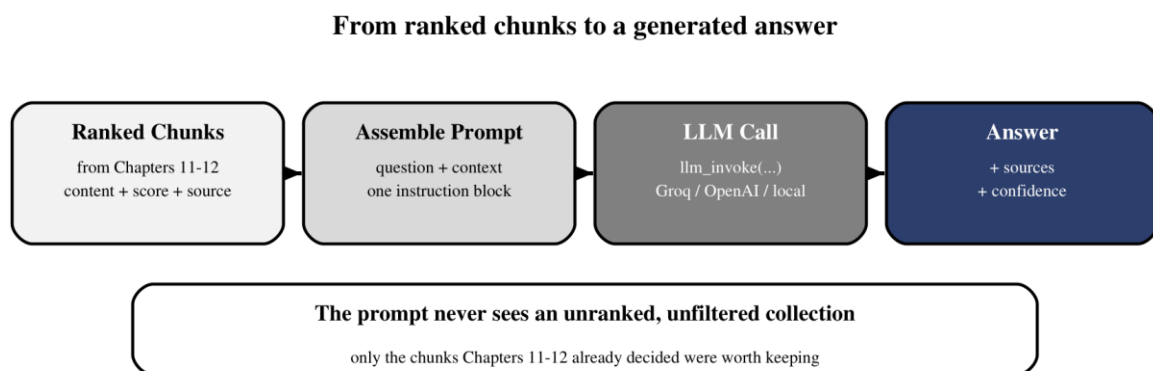


Figure 13.1 — Generation is the last of three stages; it never sees anything retrieval didn't already rank and filter.

``generate_answer`` in ``query.py`` is the smallest complete version of everything this chapter builds toward: retrieve, check for an empty result, assemble the prompt from Section 13.1, and invoke the model through the project's central ``llm_invoke`` wrapper rather than the raw LangChain client.

```

def generate_answer(query: str, retriever: RAGRetriever, llm, top_k: int) -
> str:
    retrieved_docs = retriever.retrieve(query, top_k=top_k)
    if not retrieved_docs:
        return "No relevant documents found to answer the question."

    context = "\n\n".join(
        [f"Document {doc['rank']}: \n{doc['content']}" for doc in
retrieved_docs]
    )
    prompt = f"""You are an expert assistant. Use the following retrieved
documents to answer the question.

Question: {query}

Context:
{context}

Provide a concise and accurate answer based on the above information."""

    result = llm_invoke(llm, [{"role": "user", "content": prompt}],
caller_tag="QUERY-SIMPLE")
    if not result.ok:
        return f"LLM call failed: {result.error_message}"
    return result.content

```

Notice the empty-retrieval check runs before a single token is spent on the model. A prompt built from zero chunks is not a harder question for the LLM to answer carefully — it is an invitation for the model to fill the gap from its own training data, precisely the behavior augmentation exists to prevent. Refusing to call the model at all is the correct response to no evidence, not a last resort.

13.5 Model names, deprecations, and staying current

Hosted model names are not stable identifiers — providers deprecate and rename them on their own schedule, not the project's. Memora hit this twice mid-project: both `gemma2-9b-it` and `llama3-8b-8192` were deprecated by Groq while the codebase still referenced them by name.

Common pitfall — Hardcoding a model name in application code

A model string typed directly into a `ChatGroq(...)` call becomes a bug the day the provider deprecates it — every call site needs to be found and edited, under time pressure, while the pipeline is down. Memora's fix was structural: read the model name from `.env` (`MODEL\_NAME`) with a sane default, so a provider deprecation is a configuration change, not a code change, and can be rolled out without touching `llm\_setup.py` at all.

Treat every hosted model name the way you would treat a version-pinned dependency: know where it is declared, know how to change it in one place, and expect that place to need editing eventually.

13.6 Temperature, max_tokens, and other generation knobs

Definition — Temperature

A generation parameter that scales the randomness of next-token sampling. A temperature near 0 makes the model consistently choose its highest-probability token; higher values let lower-probability tokens win more often, producing more varied output across repeated calls.

Memora does not use one temperature for every LLM role — it uses the parameter deliberately, tuned to what each call is for.

Role	Temperature	max_tokens	Why
Answer generation ('llm')	0.1	2048	Mostly deterministic prose, slight room for natural phrasing
Chunk merging ('merge_llm')	0.0	2048	A merge must be faithful, not creative — zero variance
Quality judging ('judge_llm')	0.0	1024	A judge that disagrees with itself between runs is useless

The pattern generalizes: any call whose output must be checkable, reproducible, or diffable against a prior run belongs at temperature 0. Only the final, user-facing prose has any reason to tolerate variance, and even there, 0.1 is a small concession, not an invitation for creativity.

13.7 Handling API errors, rate limits, and retries

A hosted LLM call fails more often than local code does, in ways local code rarely has to reason about: rate limits, transient server errors, connection drops, and provider-specific rejections of a well-formed request. Two incidents from Memora's own history show what happens when a pipeline treats every failure the same way — badly.

Common pitfall — Treating every LLM failure as fatal

Groq once rejected a mid-session tool-call payload with `'BadRequestError' / 'tool_use_failed'`, and the pipeline's response was a hard abort — "Unable to generate a clean answer," no retry, no fallback. Separately, an agent with no iteration cap looped over 40 times on one query, and the accumulated prompt tripped Groq's 6,000-token-per-minute limit outright (`'413: Requested 6785, Limit 6000'`). Neither failure was unrecoverable on its own; treating both as identical, fatal events was the actual bug.

The fix in both cases was the same shape: stop treating "the call failed" as one category. `'llm_invoke'` classifies every failure into an `'LLMErrorKind'` — rate limit, server error, connection, timeout, bad request, and more — and returns a typed `'LLMResult'` rather than raising or returning a bare string.

```

@dataclass
class LLMResult:
    ok: bool
    content: str = ""
    error_kind: LLMErrorKind | None = None
    error_message: str = ""

def llm_invoke(llm, messages: list, *, caller_tag: str = "LLM") ->
    LLMResult:
    ... # classify failures, retry the transient ones, return either shape

```

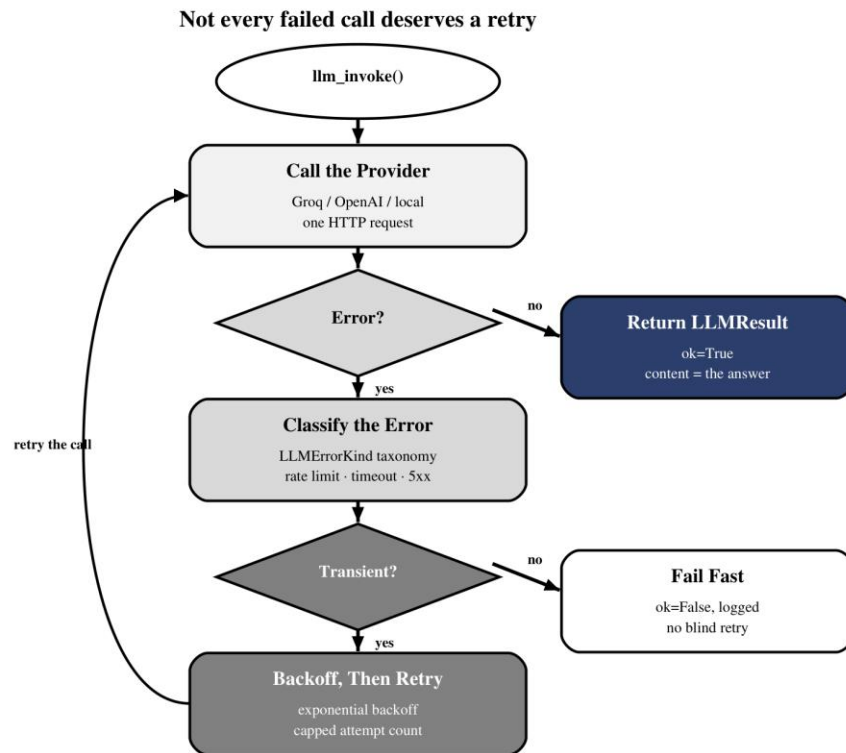


Figure 13.2 — Only transient failures earn a retry; everything else fails fast with a structured reason.

As Figure 13.2 shows, the classification decides everything downstream: a rate limit or a timeout backs off and retries, capped at a small number of attempts, while a genuinely malformed request fails immediately with a logged reason instead of retrying a call that will never succeed. Chapter 13B opens `llm_caller.py` fully — its FIFO call ordering, its adaptive cooldown derived from a provider's own rate-limit headers, and its full error taxonomy. For now, the shape to remember is simpler than the implementation: classify before you retry, and never retry a request that was wrong, only one that was unlucky.

13.8 Advanced answer formatting — sources, confidence, previews

A bare answer string is enough for a terminal demo and not enough for anything a user needs to trust. `advanced_answer` builds the same augmented prompt as Section 13.4's simple path, but returns a structured result a caller can actually display and audit.

```
def advanced_answer(query, retriever, llm, top_k,
min_score=DEFAULT_MIN_SCORE):
    retrieved_docs = retriever.retrieve(query, top_k=top_k,
score_threshold=min_score)
    if not retrieved_docs:
        return {"answer": "No relevant documents found.", "sources": [],
"confidence": 0.0}

    context = "\n\n".join(f"Document {d['rank']}: \n{d['content']}" for d in
retrieved_docs)
    sources = [
        {
            "source": d["metadata"].get("source", "Unknown"),
            "page": d["metadata"].get("page", "unknown"),
            "similarity_score": d["similarity_score"],
            "preview": d["content"][:300],
        }
        for d in retrieved_docs
    ]
    confidence = max(d["similarity_score"] for d in retrieved_docs)

    result = llm_invoke(llm, [{"role": "user", "content":
build_prompt(query, context)}], caller_tag="QUERY-ADVANCED")
    answer = result.content if result.ok else f"LLM call failed:
{result.error_message}"
    return {"answer": answer, "sources": sources, "confidence": confidence}
```

`confidence` here is deliberately cheap: the single highest similarity score among the retrieved chunks, not a second model call asking the LLM to grade itself. A retrieval-derived number is honest about what it measures — how well the evidence matched the question — instead of dressing up a model's self-assessment as if it were calibrated, which Chapter 20 will show it usually is not.

`print_advanced_result` is the last piece of this shape: a terminal-friendly renderer that walks the same dictionary and prints the answer, the confidence score, and one line per source with its filename, page, score, and a truncated preview. Nothing about the dictionary itself is terminal-specific — a web endpoint or an API response would serialize the same fields to JSON instead of `print` statements, which is exactly why `advanced_answer` returns structured data rather than a formatted string in the first place.

13.9 Streaming, citations, and conversational history — the enhanced pipeline

`query.py` answers one question at a time, with no memory of the previous turn and no partial output while the model is still generating. Both are real, common upgrades to this same augmentation step, and neither changes the core idea — only how much of the pipeline's plumbing a caller has to manage.

Streaming trades a single blocking call for a sequence of incremental chunks, useful the moment an answer takes long enough that a user benefits from seeing it arrive rather than waiting on a spinner:

```
def stream_answer(query: str, retriever, llm, top_k: int):
    context = build_context(retriever.retrieve(query, top_k=top_k))
    for chunk in llm.stream([{"role": "user", "content":
build_prompt(query, context)}]):
        yield chunk.content
```

Conversational history is the other common addition: pass prior turns as additional messages ahead of the current question, so the model can resolve "what about the second one" against something it saw. Memora's own answer to needing memory across turns is not a chat-history list, though — it is the agentic loop's run state (Chapter 16) and its persistent learned-QA collection (Chapter 11.5), which carry validated context forward deliberately rather than replaying an unfiltered transcript.